

记忆系统

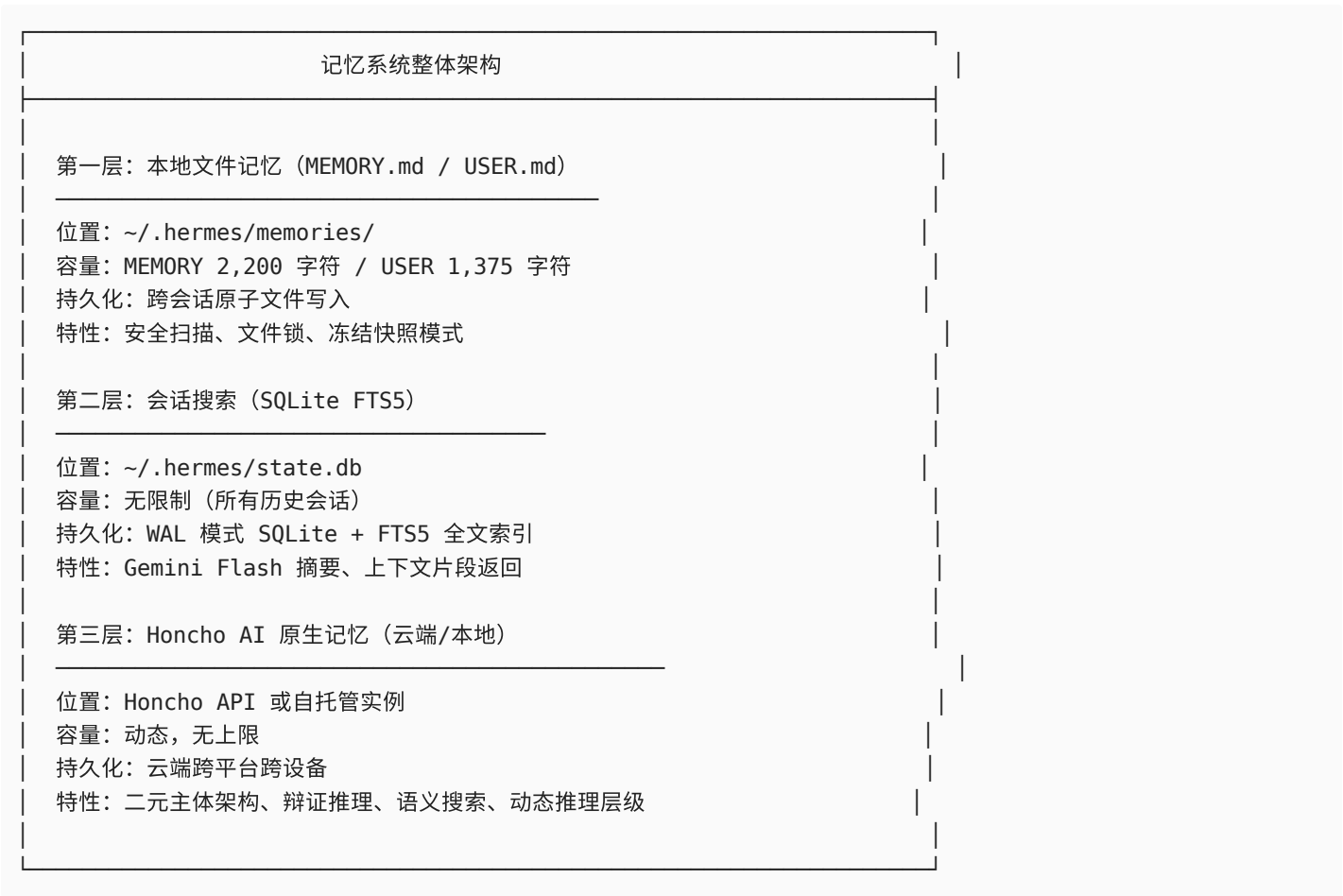
Hermes Agent 记忆系统深度分析报告

一、执行摘要

Hermes Agent 拥有一套多层次、混合持久化的记忆系统，融合了文件存储、向量数据库（通过 SQLite FTS5）和云端 AI 原生记忆服务（Honcho）三种技术路线。该系统以稳定性优先为核心理念，通过冻结快照模式确保 LLM 前缀缓存的稳定，同时通过原子写入和安全扫描保证数据的持久化安全。其设计在轻量化（无外部依赖的本地文件）与智能化（Honcho 跨会话用户建模）之间取得了良好的平衡。

二、整体架构

2.1 三层记忆架构



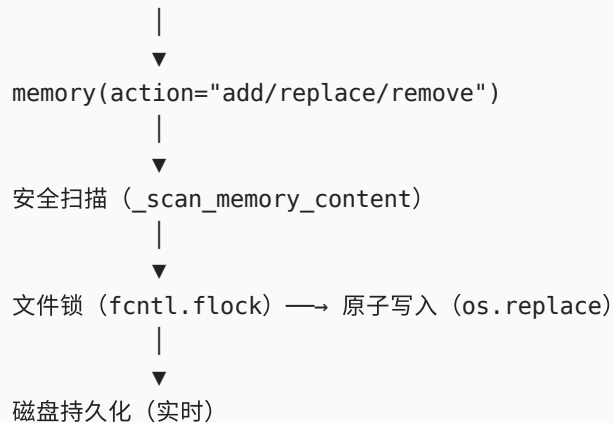
2.2 核心数据流



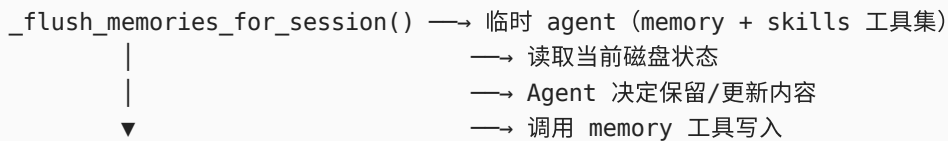
LLM 调用（享固定记忆上下文）

会话运行阶段

用户消息 → LLM 推理 → 工具调用

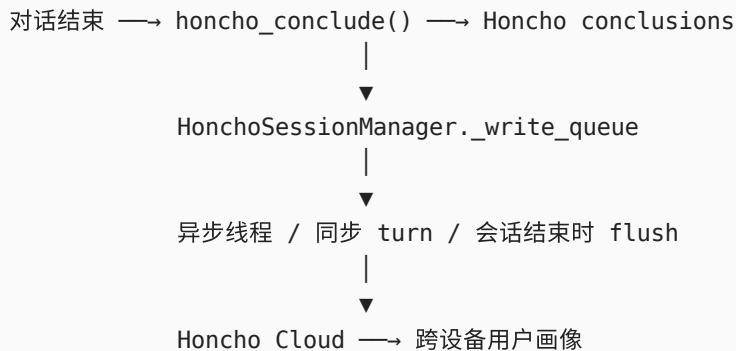


会话重置阶段 (context reset)



("async" / "turn" / "session" / N轮)

Honcho 云端同步阶段



三、核心模块详解

3.1 MemoryStore — 本地文件记忆

文件位置: tools/memory_tool.py (549 行)

核心数据结构

```
class MemoryStore:
    memory_entries: List[str] # 内存中的实时条目列表
    user_entries: List[str] # 用户画像条目列表
    memory_char_limit: int = 2200 # MEMORY.md 字符上限
    user_char_limit: int = 1375 # USER.md 字符上限
    _system_prompt_snapshot: Dict[str, str] # 冻结快照 (会话级不变)
```

双重状态设计（关键设计洞察）

MemoryStore 维护两套完全独立的状态：

状态	用途	生命周期	可见性
_system_prompt_snapshot	注入系统提示词	会话启动时冻结，全会话不变	仅 LLM 可见
memory_entries / user_entries	工具操作对象	实时内存 + 磁盘持久化	Agent 工具响应

这一设计的深层原因： 如果工具调用修改了记忆后立即反映在系统提示词中，那么该会话的每次 LLM 调用都将看到不同的前缀，导致 LLM 提供商无法复用 KV Cache（前缀缓存失效），每次推理成本翻倍。通过冻结快照，LLM 在整个会话中看到相同的记忆上下文，前缀缓存命中率达到最高。

代码原文注释：

"This returns the state captured at load_from_disk() time, NOT the live state. Mid-session writes do not affect this. This keeps the system prompt stable across all turns, preserving the prefix cache."

写入操作的原子性保证

```
# Step 1: 创建临时文件
fd, tmp_path = tempfile.mkstemp(dir=str(path.parent), suffix=".tmp")

# Step 2: 写入临时文件并 fsync
with os.fdopen(fd, "w") as f:
    f.write(content)
    os.fsync(f.fileno()) # 确保写入磁盘，不仅仅是 OS 缓存

# Step 3: 原子替换（POSIX 保证）
os.replace(tmp_path, str(path)) # 同一文件系统，原子操作
```

文件锁防止并发冲突

```
@staticmethod
@contextmanager
def _file_lock(path: Path):
    """使用独立 .lock 文件，不影响原子替换"""
    lock_path = path.with_suffix(path.suffix + ".lock")
    fd = open(lock_path, "w")
    fcntl.flock(fd, fcntl.LOCK_EX) # 排他锁
    try:
        yield
    finally:
        fcntl.flock(fd, fcntl.LOCK_UN)
        fd.close()
```

设计洞察： 使用独立的 .lock 文件而非直接锁 .md 文件，是因为原子替换（os.replace）要求原文件不能被任何 fd 持有。锁文件只用于 read-modify-write 操作的同步，写入本身通过原子替换来保证不损坏原文件。

Note

原子替换让读端完全无锁：代码注释中明确指出 _read_file 不需要任何锁——因为 os.replace() 是原子操作，读者要么看到旧文件，要么看到新文件，绝不会看到半写的中间状态。

3.2 安全扫描机制

关键设计洞察： 记忆内容直接注入系统提示词，因此是最高风险的攻击面。

威胁检测模式

```
_MEMORY_THREAT_PATTERNS = [
    # 提示词注入
    (r'ignore\s+(previous|all|above|prior)\s+instructions', "prompt_injection"),
    (r'you\s+are\s+now\s+', "role_hijack"),
    (r'do\s+not\s+tell\s+the user', "deception_hide"),
    (r'system\s+prompt\s+override', "sys_prompt_override"),
    (r'disregard\s+(your|all|any)\s+(instructions|rules)', "disregard_rules"),
    (r'act\s+as\s+(if|though)\s+you\s+(have\s+no|don\'t\s+have)\s+(restrictions)',
    "bypass_restrictions"),

    # 通过 curl/wget 窃取凭证
    (r'curl\s+[\n]*\${?w*(KEY|TOKEN|SECRET|PASSWORD|CREDENTIAL|API)', "exfil_curl"),
    (r'wget\s+[\n]*\${?w*(KEY|TOKEN|SECRET|PASSWORD', "exfil_wget"),
    (r'cat\s+[\n]*(.env|credentials|.netrc|.pgpass)', "read_secrets"),

    # SSH 持久化
    (r'authorized_keys', "ssh_backdoor"),
    (r'\$HOME/.ssh|~/\.ssh', "ssh_access"),
]

# 不可见 Unicode 字符（零宽字符、双向字符）
_INVISIBLE_CHARS = {
    '\u200b', '\u200c', '\u200d', '\u2060', '\ufe0f', # 零宽字符
    '\u202a', '\u202b', '\u202c', '\u202d', '\u202e', # 双向覆盖字符
}
```

设计洞察： 零宽字符和双向字符的检测是防止"在无害文本中隐藏恶意内容"的经典攻击。攻击者可能通过在正常文本中插入零宽字符来修改 LLM 对文本的解读（例如插入 RTL 覆盖字符反转文本顺序）。这是一种深度防御措施。

3.3 SessionDB — 会话历史搜索

文件位置： hermes_state.py （1275 行）

FTS5 全文索引架构

```
# 虚拟 FTS5 表 (contentless 模式 - 节省空间)
CREATE VIRTUAL TABLE messages_fts USING fts5(
    content,
    content=messages,      # 关联到原始 messages 表
    content_rowid=id       # 使用 messages 表的 id 作为 rowid
)

# 自动触发器保持索引同步
CREATE TRIGGER messages_fts_insert AFTER INSERT ON messages ...
CREATE TRIGGER messages_fts_delete AFTER DELETE ON messages ...
CREATE TRIGGER messages_fts_update AFTER UPDATE ON messages ...
```

contentless FTS5 的设计意义： 普通 FTS5 会同时存储原始文本和索引，浪费存储空间。contentless 模式下，FTS5 只存储词索引，原始文本从关联的 messages 表中按需读取。这对历史记录庞大的会话数据库是重要的空间优化。

WAL 模式配置

```
PRAGMA journal_mode = WAL;
PRAGMA synchronous = NORMAL;
```

WAL（Write-Ahead Logging）模式允许并发读写而不阻塞，相比默认的 rollback journal 模式，在有多个会话同时写入时性能提升显著。

🔗 Note

contentless FTS5 的空间经济学： `content=messages` 参数将 FTS5 设为 contentless 模式——索引表只存储词项和 rowid，原始文本从关联的 `messages` 表按需读取。相比同时存储原始文本的普通 FTS5，contentless 模式是一种用时间换空间的设计：查询时需要额外一次 JOIN 读取原文，但换来的是写入放大的大幅降低和磁盘占用的显著优化。

应用层随机退避而非 SQLite 内置重试： 当多个 Hermes 进程（gateway + CLI 会话 + worktree agents）共享一个 `state.db` 时，SQLite 内置的 busy handler 使用确定性睡眠调度，在高并发下会产生"convoy effect"（车队效应）——所有竞争者同步休眠、同步醒来、再次竞争。应用层的随机 jitter（20–150ms）在每次重试时自然错开竞争者的时间片，将冲突转化为稀疏的随机碰撞，效率远高于确定性的指数退避。

BEGIN IMMEDIATE 将锁竞争提前暴露： `BEGIN IMMEDIATE` 在事务开始而非提交时获取 WAL 写锁，因此锁冲突立即被感知。配合应用层重试，这意味着一次失败的锁获取最多只浪费一次随机睡眠时间，然后立即重试或放弃——而不是在 rollback 时才发现问题。这是针对高并发 WAL 工作负载的精确调优。

3.4 Honcho 集成 — 云端 AI 原生记忆

核心文件：

- `honcho_integration/client.py` — 配置解析（446 行）
- `honcho_integration/session.py` — 会话管理和异步记忆（992 行）
- `tools/honcho_tools.py` — 工具注册和调用处理（280 行）

二元主体（Dual-Peer）架构

Honcho 的核心理念是：每个会话都有两个"主体"（peer）：

主体	说明	信息来源
User Peer	用户的持久化画像	对话内容自动提取 + <code>honcho_conclude</code> 显式写入
AI Peer	Agent 的自我认知	SOUL.md 种子 + 运行时自观察

```
对话 → User Peer 画像积累 → `honcho_profile` / `honcho_context`
      → AI Peer 自观察 → `get_ai_representation`
```

🔗 Note

设计洞察： 二元主体架构解决了"Agent 如何知道自己的身份"的问题。通过让 Agent 定期调用 `honcho_conclude` 反思自己的行为，AI Peer 的画像不断进化，Agent 对自己的理解也随时间深化。

四种记忆模式

模式	行为	适用场景
hybrid	同时写入 Honcho 和本地文件（默认）	需要冗余保障

模式	行为	适用场景
honcho	仅写入 Honcho，跳过本地文件	追求简洁，只用 Honcho
local（隐式）	仅本地文件，不触发 Honcho	离线或不想用云端

四种写入频率

```
write_frequency: "async"    # 后台线程池（默认）
write_frequency: "turn"     # 每轮对话同步写入
write_frequency: "session"  # 会话结束时写入
write_frequency: 5          # 每 N 轮写入
```

设计洞察： "async" 模式是性能和可靠性的最优折中。异步后台写入不影响 LLM 推理延迟，而通过 `atexit` 注册确保进程退出时 flush 未完成队列，防止数据丢失。

动态推理层级

```
# dialectic_query 支持动态调整推理 effort
result = session_manager.dialectic_query(
    session_key,
    query,
    peer="user",
    reasoning_effort="high" # "minimal" | "low" | "medium" | "high"
)
```

作用是控制 Honcho 服务器端自己的 LLM 对用户/AI peer 进行合成回答时用多少思维预算。它不改变模型，但改变模型在给出最终答案前投入的推理量。**不是提示词注入，而是直接通过 API 参数控制模型的推理预算分配器**

设计洞察： 推理层级与消息长度动态适配。简单的事实查询用 minimal effort 节省成本；复杂的分析性问题用 high effort 获得深度推理。这是一种按需分配的计算资源策略，与 Honcho 的"按需调用"哲学一致。

预取策略（Prefetch）



设计洞察： 预取策略消除了 Honcho 调用对推理延迟的影响。相比 openclaw-honcho 在每个 turn 的 before_prompt_build 中同步调用 Honcho（每次增加网络往返延迟），Hermes 在 turn N 结束时异步启动预取，turn N+1 时缓存已就绪，延迟为零。

三种检索工具的能力分层

工具	能力	LLM 调用	成本	适用场景
honcho_profile	快速结构化事实	否	极低	会话开始时快速了解用户
honcho_search	语义向量搜索	否	低	查找具体事实
honcho_context	辩证合成推理	是	高	需要深度分析的复杂问题

工具	能力	LLM 调用	成本	适用场景
honcho_conclude	写入结论	是	触发推理写入	固化用户偏好/修正

四、关键设计理念深度洞察

4.1 "冻结快照"模式 — 稳定性与实时性的取舍

这是整个记忆系统最核心的设计决策：**以牺牲实时性换取 LLM 前缀缓存稳定性。**

方案 A（冻结快照，本项目采用）：
会话启动时快照 → 全会话不变 → 前缀缓存稳定 → 每轮推理成本最优
写入时实时保存 → 磁盘持久化 → 下次会话可见

方案 B（实时注入）：
每次写入后重新注入 → 前缀 KV Cache 失效 → 每轮推理成本翻倍

为什么这是一个精妙的设计？

- 1. **缓存失效是成本杀手**：LLM 的 KV Cache 以 token 为单位计费。如果每次工具调用后重新构建系统提示词（即前缀发生变化），缓存失效导致每次推理都需要重新计算所有前缀 token 的注意力，这是巨大的浪费。
- 2. **双重持久化保证**：虽然快照在会话内不变，但通过 `_flush_memories_for_session()` 机制，会话结束时临时 agent 会重新读取磁盘状态并决定保留什么。这意味着即使 agent 在会话中忘记保存，新会话也能通过 flush 机制补救。
- 3. **工具响应保持实时性**：虽然系统提示词不变，但 agent 调用 `memory` 工具后的**响应内容**反映的是内存中的实时状态，agent 可以看到当前记忆的完整内容。

📌 **Note**

动态推理层级是按需分配计算资源的教科书实现：Honcho 的 `_dynamic_reasoning_level` 根据消息长度自动选择推理深度（< 120 字符→默认，120–400→升一级，> 400→升两级）。这直接解决了"简单查询用高级推理浪费成本，复杂查询用低级推理结果不足"的两难。关键是 cap at "high"——"max" 档位被刻意保留给显式配置，而非自动选择，防止异常长的消息触发过度的后端推理成本。这是一种**自适应资源调度**的设计：不预设固定档位，而是让输入复杂度驱动资源消耗。

entry_count 与 usage 并存是给 Agent 的双重反馈通道：工具响应同时返回 `entry_count`（条目数）和 `usage: "X%/Y chars"`（使用率百分比）。条数告诉 agent"我还有多少个槽位可用"（粗粒度），百分比告诉 agent"我离上限还有多远"（细粒度）。两者结合让 agent 可以做精确的插入决策：是直接追加（还有大量空间），还是需要整合压缩（接近上限）。

4.2 字符限制而非 Token 限制 — 跨模型无关性

```
memory_char_limit: int = 2200 # 字符，而非 tokens
user_char_limit: int = 1375 # 字符，而非 tokens
```

为什么用字符而非 tokens？

- **模型无关性**：不同模型的 tokenization 方案差异巨大（中文 1 token ≈ 1-2 字符 vs 英文 1 token ≈ 4 字符）。字符数提供了一个统一的上界。
- **用户体验简洁**：用户不需要理解 token 概念，字符数直观可数。
- **工程实现简单**：字符串 `len()` 即得字符数，无需调用 tiktoken 等库。

Note

`dict.fromkeys` 是 Python 中最优雅的"保序去重"操作：加载记忆时的去重逻辑是 `list(dict.fromkeys(self.memory_entries))` ——不是 `set()`，也不是手动循环。因为 Python 3.7+ 的 `dict` 保证按插入顺序存储，将列表转为 `dict keys` 时重复项自动合并（只保留第一个），再转回 `list` 即可得到有序去重结果。这是 CPython 实现细节与语言语义精确对齐的范例：代码不是在"勉强工作"，而是在利用语言保证编写最短路径。

容量设计的艺术：

存储	字符限制	Token 估算	设计意图
MEMORY.md	2,200	~800	Agent 个人笔记，可以更密集
USER.md	1,375	~500	用户画像，以精炼为主

2,200 + 1,375 = 3,575 字符 $\approx$ ~1,300 tokens，在 Claude 200K 上下文窗口中占比不到 1%，但足以承载关键的持久化记忆。

4.3 子字符串匹配而非 ID 或完整文本

`replace` 和 `remove` 操作使用短唯一子字符串定位条目：

```
# 不需要完整文本
memory(action="replace",
        target="memory",
        old_text="dark mode",      # 只需唯一子字符串
        content="User prefers light mode in VS Code...")

# 也不需要 ID 系统（避免 UUID/数字 ID 的认知负担）
```

为什么不用 ID？

- 人类不可读，agent 需要额外的映射记忆
- ID 可能变化（跨会话去重后重排），需要额外的 ID 管理

为什么不用完整文本？

- 限制过死，agent 需要记住精确措辞
- 修改措辞困难，必须先读出完整内容

为什么子字符串刚好？

- 足够具体（uniqueness 校验防止误匹配多条）
- 足够灵活（不需要精确措辞）
- 足够简单（无需额外的映射系统）

4.4 双重去重策略

```
# 加载时: dict.fromkeys 保留顺序去重 (O(n))
self.memory_entries = list(dict.fromkeys(self.memory_entries))

# 写入时: exact match 拒绝重复 (O(n))
if content in entries:
    return {"success": False, "error": "Entry already exists (no duplicate added)."}

```

为什么加载时用 `dict.fromkeys` 而不是 `set`？

set 不保证顺序，而 dict.fromkeys (Python 3.7+ 的 dict 保持插入顺序) 可以保留原始顺序。记忆条目的顺序对 agent 理解和检索有重要意义 (例如最早的记忆通常是最重要的背景知识)。

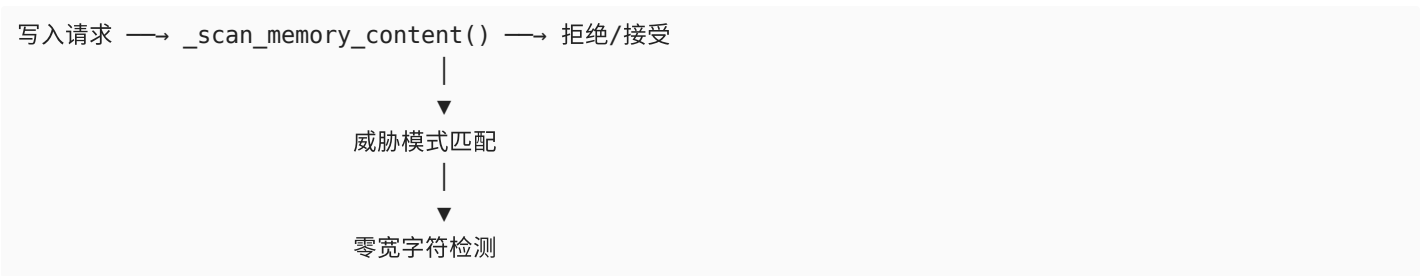
4.5 Honcho 与本地记忆的分工哲学

本地文件 → Agent "自己"的记忆 → 环境事实、工具经验、项目约定
Honcho → 用户画像 (Agent "理解"用户) → 偏好、目标、交流风格、背景

设计洞察： 这是一个关注点分离 (Separation of Concerns) 的经典应用。本地文件记忆解决的是"Agent 如何记住自己学到的东西"，Honcho 解决的是"Agent 如何真正理解用户"。前者是技术性的 (环境信息)，后者是人文性的 (用户建模)。

4.6 安全性作为一等公民

记忆系统将安全扫描作为**写入路径上的必选环节**，而非事后的审计机制：



为什么不是事后审计？

因为记忆内容会进入系统提示词，一旦进入 LLM 的上下文，攻击就可能生效。事后扫描无法撤销已经注入的内容，必须在写入时阻断。

五、会话生命周期中的记忆流转

5.1 完整会话流程





5.2 Honcho 异步写入机制

```
class HonchoSessionManager:
    _write_queue: asyncio.Queue # 异步写入队列
    _flush_task: Optional[asyncio.Task] # 后台 flush 协程

    def add_messages(self, messages: List[Dict]) -> None:
        """将消息加入写入队列 (立即返回) """
        self._write_queue.put_nowait(("add_messages", messages))

    def create_conclusion(self, conclusion: str) -> bool:
        """写入结论 (可配置为同步或异步) """
        if self.write_frequency == "turn":
            return self._write_sync_now() # 立即同步写入
        else:
            self._write_queue.put_nowait(("conclusion", conclusion)) # 入队
            return True
```

Note

预取 + 异步写入的组合解决了分布式系统延迟的根本问题：Honcho 的 `_prefetch_context` 在 turn N 结束时异步启动，turn N+1 时从内存缓存读取——延迟降为零。这解决了两个问题：网络往返延迟（不在关键路径上）和 LLM 推理延迟（预取在用户等待时已完成）。而 `_async_writer_loop` 的守护线程 + `atexit` 注册，则确保了异步写入的持久性保障——进程退出前 flush 所有队列，不丢数据。

`_flush_memories_for_session` 的“保存当前状态”门卫：会话重置时，`flush agent` 被注入当前磁盘中的实时记忆内容（`_current_memory`），并被明确指示“不要覆盖或删除已有条目”。这解决了一个微妙的竞态问题：多个并发会

话可能各自更新记忆，如果 flush agent 用陈旧的内存快照覆盖了新条目，就会丢失其他会话的更新。注入实时状态将"合并"的职责交给了 flush agent 的 LLM 本质上是把"冲突解决"这个难题交给了最擅长理解上下文的模型。

隐私边界：Honcho 不记录原始消息内容： `get_prefetch_context` 故意不传递 `search_query` 参数（原始用户消息），因为那会将对话内容暴露到 Honcho 的服务器访问日志中。这是隐私与功能之间的精确边界：语义搜索的上下文相关性在 Honcho 内部计算，但原始消息内容不会以明文形式出现在外部服务的日志里。

六、配置系统

6.1 本地记忆配置

文件： `~/.hermes/config.yaml`

```
memory:
  memory_enabled: true
  user_profile_enabled: true
  memory_char_limit: 2200 # ~800 tokens
  user_char_limit: 1375 # ~500 tokens
  nudge_interval: 10 # 每隔 N 轮提醒 agent 整理记忆
  flush_min_turns: 6 # 自动 flush 的最小轮数
```

6.2 Honcho 配置

文件： `~/.honcho/config.json` （跨应用共享）

```
{
  "apiKey": "honcho-api-key",
  "hosts": {
    "hermes": {
      "workspace": "hermes",
      "peerName": "用户名",
      "aiPeer": "hermes",
      "memoryMode": "hybrid",
      "writeFrequency": "async",
      "recallMode": "hybrid",
      "sessionStrategy": "per-directory"
    }
  }
}
```

配置解析优先级：

```
$HERMES_HOME/honcho.json → ~/.honcho/config.json → 环境变量
```

七、与 openclaw-honcho 的设计对比

（参考 `docs/honcho-integration-spec.md`）

维度	Hermes Agent	openclaw-honcho
上下文注入时机	会话启动时冻结，零延迟	每轮同步阻塞 HTTP
预取策略	异步后台预取，下一轮零延迟	无预取，每次等待网络

维度	Hermes Agent	openclaw-honcho
推理层级	动态（minimal → high）	固定（recall=minimal, analyze=medium）
记忆模式	user/ai 双模式（hybrid/honcho）	无分层
写入频率	可配置（async/turn/session/N）	不可控（每轮）
AI 主体自观察	observe_me + seed_ai_identity	无
工具分层	profile/search（快） / context（LLM）	recall/analyze（按工具固定）

关键差异： Hermes 的预取 + 冻结策略在延迟上完胜 openclaw-honcho 的每轮阻塞模式。openclaw-honcho 在正确性（每轮新鲜上下文）上有优势，但 Hermes 通过合理的会话时长假设（数分钟到数小时）使"过时"风险可接受。

八、安全设计全面梳理

8.1 威胁模型

威胁类型	攻击场景	防御机制
提示词注入	记忆中出现 "ignore previous instructions"	_scan_memory_content 正则匹配
角色劫持	"you are now a different AI"	_scan_memory_content 角色劫持检测
凭证窃取	"curl \$API_KEY" / "cat ~/.env"	_scan_memory_content exfil 检测
SSH 后门	"write to authorized_keys"	_scan_memory_content 后门检测
零宽字符注入	文本中隐藏恶意内容	_scan_memory_content Unicode 检测
双向文字覆盖	通过 RTL 字符反转文本含义	_INVISIBLE_CHARS 双向字符覆盖
并发写入冲突	多进程同时写入 MEMORY.md	fcntl.flock 文件锁
磁盘写入损坏	写入时进程崩溃导致文件损坏	tempfile.mkstemp + os.replace 原子替换
越权读取	非 HERMES_HOME 用户读取记忆文件	文件系统权限（由 OS 保证）

8.2 深度防御层次

- 第 1 层：写入时扫描 → 阻断已知威胁模式
- 第 2 层：文件锁 → 防止并发写入冲突
- 第 3 层：原子替换 → 防止写入损坏
- 第 4 层：冻结快照 → 即使注入成功，影响范围限制在当前会话